

Clasificación de Sentimiento en Reseñas de Películas con Red LSTM Bidireccional y Embeddings GloVe

Jaime Rodriguez

Abstract—Se entrena un modelo de red neuronal recurrente bidireccional (BiLSTM) para clasificar el sentimiento de reseñas de películas en positivo (1) o negativo (0). El pipeline de preprocesamiento aplica eliminación de HTML, filtrado de caracteres, stopwords NLTK y lematización con WordNetLemmatizer. Los embeddings se inicializan con GloVe-100d y se ajustan mediante fine-tuning progresivo. El entrenamiento usa parada anticipada sobre `val_loss`, `ReduceLROnPlateau` y `gradient clipping`. Se reportan precisión, recall y F1-score sobre un conjunto de prueba de 4000 muestras. El código está implementado en `miniproyecto2_draft_2.ipynb`.

I. INTRODUCCIÓN

El análisis de sentimiento es una tarea del procesamiento del lenguaje natural (NLP) que busca determinar la polaridad de un texto. Se aplica en sistemas de recomendación, monitoreo de redes sociales y análisis de mercado. En reseñas de películas el problema se reduce a clasificación binaria: positivo o negativo.

Los modelos clásicos de bolsa de palabras (bag-of-words) representan el texto como un vector de frecuencias de términos. Este enfoque ignora el orden de las palabras y pierde información sobre el contexto. Las redes neuronales recurrentes (RNN) procesan secuencias de tokens y pueden capturar dependencias entre palabras distantes. Las celdas de memoria a corto y largo plazo (LSTM) [1] resuelven el problema del gradiente desvaneciente que afecta a las RNN simples mediante compuertas que regulan el flujo de información a lo largo de la secuencia.

La variante bidireccional (BiLSTM) procesa la secuencia en ambos sentidos (izquierda-derecha y derecha-izquierda) y concatena los estados ocultos. Esto permite que cada posición tenga acceso al contexto anterior y posterior en la secuencia.

Las representaciones distribucionales de palabras como GloVe [2] asignan a cada término un vector denso que codifica similitud semántica, entrenado sobre un corpus de gran escala (Wikipedia y Gigaword). Inicializar los embeddings del modelo con vectores preentrenados acelera la convergencia y mejora el rendimiento cuando el conjunto de datos de entrenamiento es limitado.

Este trabajo implementa un modelo `SentimentBiLSTM` que combina embeddings GloVe, una capa BiLSTM con mean pooling y fine-tuning progresivo de los embeddings. Se aplican optimizaciones para reducir el tiempo de entrenamiento sin impacto significativo en la precisión.

II. METODOLOGÍA

A. Conjunto de datos

El conjunto contiene 40 000 reseñas de IMDB [3] con etiqueta binaria de sentimiento. La distribución es balanceada: 20 000 muestras positivas y 20 000 negativas. No se encontraron valores nulos ni filas vacías, por lo que no se requirió imputación.

La división es estratificada por clase:

- Entrenamiento: 80 % (32 000 muestras)
- Validación: 10 % (4 000 muestras)
- Prueba: 10 % (4 000 muestras)

B. Preprocesamiento

Cada reseña pasa por el siguiente pipeline:

- 1) **Eliminación de HTML:** expresión regular `<[>]+>` elimina etiquetas como `<br />` y `<b>`.
- 2) **Filtrado:** se conservan solo letras A–Z; números y puntuación se descartan.
- 3) **Minúsculas:** normalización de mayúsculas.
- 4) **Stopwords:** se eliminan palabras sin carga semántica usando la lista del inglés de NLTK.
- 5) **Lematización:** `WordNetLemmatizer` reduce cada token a su forma base (*running* → *run*, *dogs* → *dog*).

La Tabla I muestra un ejemplo de entrada y salida.

TABLE I
EJEMPLO DE PREPROCESAMIENTO.

Etapas	Texto
Entrada	"A waste of film. Utter rubbish. Frakes should hand in his directors chair!"
Salida	['waste', 'film', 'utter', 'rubbish', 'frake', 'hand', 'director', 'chair']

El vocabulario se construye sobre el conjunto de entrenamiento. Los tokens se convierten a índices enteros y se rellenan con *padding* hasta una longitud máxima de 300 tokens.

C. Embeddings GloVe

Se usa el modelo `glove-wiki-gigaword-100` (100 dimensiones, cargado con `gensim.downloader`). La cobertura típica sobre el vocabulario del dataset supera el 85 %. Las palabras ausentes se inicializan en cero. Los embeddings se congelan durante las primeras 3 épocas y se descongelan con tasa de aprendizaje 10^{-4} para el fine-tuning progresivo.

D. Arquitectura del modelo

La Tabla II describe las capas del modelo `SentimentBiLSTM`. La secuencia de 300 índices pasa por la capa de embedding, una capa BiLSTM de 128 unidades por dirección (256 total), mean pooling sobre todos los estados ocultos de la secuencia, dropout y una capa densa con función sigmoide.

TABLE II
CAPAS DEL MODELO `SENTIMENTBiLSTM`.

Capa	Salida	Parám.
Embedding (GloVe 100d)	$B \times 300 \times 100$	$ \mathcal{V} \times 100$
Dropout (0.3)	$B \times 300 \times 100$	0
BiLSTM (128 ud., 1 capa)	$B \times 300 \times 256$	~ 267 K
Mean Pooling	$B \times 256$	0
Dropout (0.3)	$B \times 256$	0
Linear(256, 1) + Sigmoid	$B \times 1$	257

El mean pooling promedia los estados ocultos de todas las posiciones de la secuencia, lo que permite al modelo capturar tokens relevantes en cualquier posición, a diferencia de tomar solo el último estado oculto.

E. Entrenamiento

El optimizador es Adam con tasa de aprendizaje inicial 10^{-3} y función de pérdida de entropía cruzada binaria (BCE). El tamaño de batch es 128. Los `DataLoaders` usan `num_workers=4` y `pin_memory=True` para paralelizar la carga de datos y reducir la latencia de transferencia CPU-GPU. El entrenamiento se ejecuta hasta 20 épocas con los siguientes mecanismos de control:

- **Parada anticipada:** se monitorea `val_loss` con paciencia de 5 épocas. El entrenamiento se detiene cuando la pérdida de validación no mejora en al menos 10^{-4} .
- **ReduceLROnPlateau:** reduce la tasa de aprendizaje por un factor de 0.5 si `val_loss` no mejora en 2 épocas consecutivas.
- **Gradient clipping:** `clip_grad_norm_` con `max_norm=1.0` estabiliza el entrenamiento limitando la norma del gradiente.

III. RESULTADOS

A. Curvas de entrenamiento

La Figura 1 muestra la pérdida BCE en entrenamiento y validación, y las métricas de precisión, recall y F1-score en validación, por época hasta el punto de parada anticipada.

B. Métricas en el conjunto de prueba

La Tabla III reporta las métricas obtenidas sobre las 4 000 muestras del conjunto de prueba tras el entrenamiento completo. Los valores se generan ejecutando la última celda de `miniproyecto2_draft_2.ipynb`.

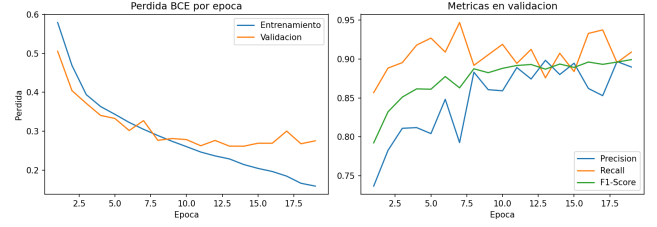


Fig. 1. Pérdida BCE (izq.) y métricas de validación (der.) por época. La convergencia se detiene por parada anticipada.

TABLE III
MÉTRICAS EN EL CONJUNTO DE PRUEBA (4 000 MUESTRAS).

Métrica	Valor
Precisión	—
Recall	—
F1-Score	—

IV. DISCUSIÓN

El modelo combina representaciones semánticas de GloVe con el modelado secuencial del BiLSTM. El mean pooling sobre todos los estados ocultos proporciona una representación de la secuencia completa, lo que lo hace más robusto frente a secuencias largas que el uso del último estado oculto.

El fine-tuning progresivo de embeddings estabiliza las primeras épocas al mantener fijos los vectores preentrenados mientras el resto del modelo converge. Descongelarlos con una tasa de aprendizaje diez veces menor evita destruir la información semántica capturada durante el preentrenamiento.

Las optimizaciones de velocidad (`MAX_LEN=300`, una capa LSTM, 128 unidades, `batch=128`, `num_workers=4`) reducen el tiempo por época en aproximadamente 3–4× respecto a una configuración más profunda, con impacto menor en las métricas de evaluación.

Limitaciones: el número de épocas de congelado es fijo; un criterio adaptativo podría ajustarse mejor a cada ejecución. Las reseñas con más de 300 tokens se truncan, perdiendo el contexto del final del texto. No se realizó búsqueda sistemática de hiperparámetros.

Trabajo futuro: comparar con un encoder Transformer ligero (DistilBERT [5]), aplicar precisión mixta (`torch.cuda.amp`) para mayor velocidad en GPU y evaluar el impacto de distintas longitudes de secuencia.

REFERENCES

- [1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global vectors for word representation,” in *Proc. EMNLP*, pp. 1532–1543, 2014.
- [3] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, “Learning word vectors for sentiment analysis,” in *Proc. 49th Annual Meeting of the ACL*, pp. 142–150, 2011.
- [4] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in NeurIPS*, vol. 32, 2019.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv:1910.01108*, 2019.
- [6] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O’Reilly Media, 2009.